Etude de Conformité RGPD et Sécurité

1.Analyse des risques sur la BDD initiales :

Problématique : Pourquoi le fait de stocker les emails en clair dans la table customer_transactions posait-il un problème majeur d'un point vu de la sécurité et de la législation ?

Stocker les adresses mails en clair dans la table customer_transactions pose un problème important pour plusieurs raisons, à la fois pour la sécurité mais aussi pour la loi.

Nous allons évoquer point par point les raisons pour lesquelles cette décision et problème majeur.

Dans un premier cas, cela peut poser ***un problème de sécurité.***

Si les emails sont enregistrés en clair dans la base de données, toute personne qui accède à la base peut les lire facilement. En cas de piratage ou de fuite de données, les attaquants peuvent récupérer toutes les adresses mail des clients. Ces adresses peuvent être utilisés pour envoyer des spam, faire du phishing ou essayer d'arnaquer les utilisateurs.

De plus l'adresse mail est considéré comme une donnée personnelle qui est régie par ***les lois du Règlement Général sur la Protection des Donnée Français (RGPD).*** Les entreprises ont le devoir de protéger les données personnelles de leurs utilisateurs si cette condition n'est pas respect l'entreprise, les autorités de protection des données comme la Commission Nationale de l'informatique et des libertés démarrerons une démarche pour que l'entreprise encourt une peine pouvant aller de 5 d'emprisonnement a 300 000 €.

Toutefois garder les mails dans une table transactions n'est pas nécessaire c'est même une faute professionnelle grave de la part des développeurs. Il est souvent préférable d'utiliser un identifiant client plutôt que l'email directement. Ça limite les informations personnelles stockées dans plusieurs endroits de la base de données.

2.Principe de minimisation des données :

Le Règlement Général sur la Protection des Données (RGPD) impose aux organisations de ne collecter et traiter que les données strictement nécessaires à l'objectif du traitement. Ce principe est appelé minimisation des données. Dans le cadre du projet NovaRetail, le pipeline ETL développé en Java avec Apache Spark applique ce principe lors des différentes étapes de traitement des données.

Lors de la phase d'extraction, les données sont récupérées depuis la base PostgreSQL (choix technique optimisé remplaçant MySQL) via une connexion JDBC afin d'être traitées par le moteur Spark.

Ensuite, une étape de nettoyage des données est appliquée. Les transactions contenant des informations corrompues ou incomplètes, notamment celles dont la colonne country est NULL, sont automatiquement supprimées du DataSet. Cela permet de ne conserver que des données exploitables.

Le pipeline applique également une étape d'anonymisation. La colonne customer_email, qui contient une information personnelle permettant d'identifier directement un utilisateur, est supprimée du DataSet final. Cette suppression réduit fortement les risques liés à l'exposition des données personnelles.

Ainsi, seules les informations nécessaires aux analyses commerciales sont conservées, telles que :

- Le pays du client
- L'âge du client
- Le montant de l'achat
- Le niveau d'accès du compte

Grâce à ces transformations, le pipeline limite la quantité de données personnelles stockées dans le système et respecte ainsi le principe de minimisation des données imposé par le RGPD.

Cas d'usage : Suppression des données d'un client

Le RGPD accorde aux utilisateurs un droit à l'effacement, également appelé droit à l'oubli, qui permet à un client de demander la suppression complète de ses données personnelles.

Dans l'architecture mise en place, les données transformées sont stockées dans un Data Lake sous forme de fichiers JSON. Ces fichiers sont organisés grâce à un système de partitionnement par pays utilisant la fonction : **partitionBy("country")**

Cela signifie que les données sont stockées dans des dossiers distincts selon le pays des clients.

Si un client espagnol demande la suppression de ses données, NovaRetail peut directement accéder au dossier correspondant :

country=Espagne

Cette organisation permet de retrouver rapidement les données concernées sans devoir analyser l'ensemble du Data Lake.

Une fois les fichiers localisés, un nouveau traitement Spark peut être exécuté afin de filtrer les enregistrements liés à ce client, puis réécrire les fichiers mis à jour dans la partition concernée.

Ce mécanisme permet donc de répondre efficacement aux demandes de suppression des données imposées par le RGPD.

3. Sécurité et architecture :

Mesures de sécurité recommandées pour protéger les fichiers JSON :

Afin de garantir la sécurité des données exportées dans le Data Lake, plusieurs mesures de sécurité doivent être mises en place pour protéger les fichiers JSON stockés sur les serveurs de l'entreprise.

Tout d'abord, il est essentiel de mettre en place une gestion stricte des droits d'accès. Seuls les utilisateurs ou services autorisés doivent pouvoir accéder aux dossiers contenant les données. Cette gestion peut être réalisée à l'aide de permissions sur le système de fichiers ou via un système de gestion des identités.

Ensuite, il est recommandé de mettre en place un chiffrement des données. Le chiffrement permet de protéger les informations stockées en cas d'accès non autorisé aux serveurs ou en cas de fuite de données.

Il est également important de mettre en place des journaux d'accès (logs) permettant de tracer les accès aux fichiers. Ces journaux permettent de surveiller les activités et de détecter rapidement toute tentative d'accès suspecte.

Enfin, les serveurs hébergeant le Data Lake doivent être protégés par des mesures de sécurité réseau, telles que des pares-feux, des systèmes d'authentification sécurisés et des mises à jour régulières des systèmes afin de corriger les vulnérabilités.

Ces différentes mesures permettent de garantir un niveau de sécurité élevé pour les données archivées et de réduire les risques liés aux cyberattaques ou aux accès non autorisés.

4.Gérer les accès et la sécurité sur l'Architecture :

Dans une architecture Big Data, les données sont réparties sur plusieurs serveurs et plusieurs machines. Cela signifie que la sécurité doit être gérée sur un grand nombre de systèmes en même temps. Il faut donc contrôler les accès sur chaque machine, ce qui rend la gestion plus difficile.

Ces systèmes utilisent aussi plusieurs outils et composants différents qui travaillent ensemble. Chaque outil peut avoir ses propres règles de sécurité et ses propres paramètres d'accès. Il faut donc configurer et surveiller plusieurs systèmes en même temps, ce qui complique la gestion globale de la sécurité.

Il y a également beaucoup plus de données et d'utilisateurs dans les environnements Big Data. Cela augmente le risque d'erreurs de configuration ou d'accès non autorisés si les permissions ne sont pas correctement gérées.